

Claude Code, заточенный *под вас*.

До этого мы ставили чужие *skills* и *MCP*. Сегодня делаем свои: *skill* «персональный CFO», который по одной команде приносит финансовый отчёт; *slash*-команду `/прогноз`; *hook*, считающий рабочее время; и настраиваем поведение *Claude* через `settings.json` и *output styles*.

ДЛИТЕЛЬНОСТЬ

7–9 дней · 2–4 ч/день

ЧТО ВЫ ОСВОИТЕ

свои **skills** · **slash**-команды · **hooks** · **output styles** · **settings.json**

ГЛАВНЫЙ ИТОГ

Claude Code превращается в *ваш* инструмент

АРТЕФАКТ

skill «персональный CFO», команда `/прогноз`, **hook** рабочего времени, свой **output style**

Большинство фич этапа встроены. Один skill полезен для дисциплины написания собственных skills, ещё один – для настройки клавиатуры.

Оба опциональны.

ОБЯЗАТЕЛЬНО

Из этапов 00–07

- Claude Code, MCP `@playwright/mcp` и `@upstash/context7-mcp`;
- пакет skills `superpowers` (для skill-creator и hook-development);
- проект `portfolio-journal` или личный инвест-кабинет – используем как «реальные данные» для своего skill.

ЖЕЛАТЕЛЬНО • SKILL

skill-creator

Что это: skill, который учит Claude правильно *создавать другие skills*: какую структуру файлов делать, как писать описание (для authentic триггеринга), как организовать ссылки на дополнительные ресурсы (progressive disclosure).

Зачем сейчас: для первого проекта (skill «персональный CFO»). Без skill-creator Claude может сделать рабочий, но не «самозапускающийся» skill – пользователю придётся вспоминать, как его звать.

Где взять: smithery.ai/skills/anthropics/skill-creator

```
$ npx @smithery/cli@latest skill add anthropics/skill-creator --agent claude-code
```

update-config

Что это: skill для управления настройками Claude Code через `settings.json`.

Учит Claude добавлять разрешения, переменные окружения, hooks без ручной правки JSON.

Зачем сейчас: когда настраиваете hooks или output styles, проще сказать Claude «добавь hook на PreToolUse для Bash», чем редактировать конфиг руками.

Устанавливать не нужно: встроен в Claude Code (как `claude-api` в этапе 06).

Проверка — `/skills`.

keybindings-help

Что это: skill для настройки клавиатурных сокращений в Claude Code (через

`~/.claude/keybindings.json`).

Зачем сейчас: на этапе тренируется привычка «своё — быстрее»: свои сокращения для частых действий ускоряют работу.

Устанавливать не нужно: встроен в Claude Code, как `update-config`.

Проверка — `/skills`.

Slash-команды и hooks ставить не нужно — они просто файлы в `.claude/commands/` и `.claude/hooks/` вашего проекта (или в `~/.claude/` для глобальных). Claude сам всё создаст.

Свои skills: рабочая инструкция, которую Claude сам подхватит

До этого мы ставили чужие skills через Smithery. Они живут в `~/.claude/skills/` или в `.claude/skills/` внутри проекта. Свой skill — это *такая же папка*, только сделанная вами.

Минимальная структура skill:

```
.claude/skills/personal-cfo/
├── SKILL.md           # главный файл с описанием и логикой
├── scripts/           # вспомогательные скрипты (опционально)
│   └── compute_kpi.py
├── templates/         # шаблоны (опционально)
│   └── monthly-report.md
└── references/        # доп. знания (опционально)
    └── tax-rates.md
```

Главный файл — `SKILL.md`. У него фронтматтер с двумя обязательными полями:

```
---
name: personal-cfo
description: Используется, когда пользователь просит финансовый отчёт.
  Например: "сделай отчёт за апрель", "посчитай savings rate",
  "сравни траты с прошлым кварталом". Анализирует
  data/operations.csv и выдаёт markdown-отчёт.
---

# Personal CFO

## Когда использовать
- запрос финансового отчёта за период;
- ...

## Структура отчёта
...
```

Главное про `description` : это «триггер» skill. Claude читает `description` и *сам решает*, когда применить skill, без явной команды от вас. Чем точнее `description` (примеры формулировок, ключевые слова, контексты) – тем лучше срабатывает.

Skill `skill-creator` учит Claude писать `description` правильно. Без него вы получите skill, который придётся вызывать словами «используй skill `personal-cfo`». С ним – «сделай отчёт за апрель», и Claude сам подхватит skill.

Slash-команды: шорткаты для частых задач

Slash-команды – это просто файлы в `.claude/commands/` (для проекта) или `~/.claude/commands/` (глобально). Один файл – одна команда.

Файл `.claude/commands/прогноз.md` :

```
---
description: Прогноз сбережений с тремя сценариями
allowed-tools:
  - Bash
  - Read
  - Write
  - WebFetch
---

Сделай прогноз сбережений за период из аргументов.

Аргументы: $ARGUMENTS

Шаги:
1. Прочитай data/operations.csv;
2. Вычисли текущий капитал и месячный денежный поток;
3. Проиграй три сценария (consrv 5%, real 8%, agg 12%);
4. Запиши результат в reports/forecast-$(date +%Y-%m-%d).md;
5. Открой через playwright и сделай скриншот.
```

После этого в диалоге работает `/прогноз 30 лет` – Claude применит инструкцию из файла, подставив `30 лет` вместо `$ARGUMENTS`.

Что важно:

- **Slash-команда** – это не **skill**. Команда выполняется только когда вы её вызвали (`/прогноз`). Skill вызывается *автоматически* по triggers в description.
- **Slash-команды для проектных рутин** кладут в `.claude/commands/` и коммитят в git. Команды для всех ваших проектов – в `~/.claude/commands/`.
- **allowed-tools** – список инструментов, которыми команда может пользоваться. Полезно для безопасности: команда «отчёт» не должна иметь право удалять файлы.

Hooks: автоматические действия в ответ на события

Hook – это команда (или скрипт), которую Claude Code запускает автоматически в ответ на определённое событие. Настраивается в `~/.claude/settings.json` (глобально) или `.claude/settings.json` (проектно).

Главные события:

- **PreToolUse** – перед использованием инструмента (Bash, Edit, Write, и т. д.). Можно *заблокировать* опасную команду до её запуска.
- **PostToolUse** – после использования. Удобно для логирования.
- **Stop** – когда Claude закончил отвечать. Можно показать уведомление или открыть результат.
- **SessionStart** – при старте сессии. Можно прогреть контекст полезной информацией.
- **SessionEnd** – при выходе. Хорошее место для ведения timesheet.
- **UserPromptSubmit** – перед обработкой вашего сообщения. Можно перехватить или дополнить контекст.
- **PreCompact** – перед `/compact`. Можно сохранить состояние.

Простой hook – «лог запуска любой Bash-команды»:

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "echo \"$(date +%Y-%m-%d_%H:%M) bash\" >> ~/.claude/"
          }
        ]
      }
    ]
  }
}
```

Более полезный hook – «защита от утечки секретов»: перед каждым `git commit` сканировать staged-файлы на ключи, и если есть – блокировать.

Главное правило: **hooks выполняются автоматически**. Если поставите hook, который занимает 5 секунд – каждый запуск Bash будет тормозить на 5 секунд. Делайте hooks быстрыми.

Output styles: меняем «голос» Claude

Claude Code умеет переключаться между «стилями ответа». Встроенные:

- **default** – обычный, лаконичный. Делает задачу, без лишних объяснений.
- **explanatory** – с вставками «★ Insight» по ходу – объясняет важные решения и нюансы кода. Для учёбы.
- **learning** – ещё медленнее: задаёт вам вопросы, оставляет места «доделать самому», помогает учиться через практику.

Переключаются через `/output-style`.

Можно сделать **свой** output style. Файл `~/.claude/output-styles/cfo.md`:

```
---  
name: cfo  
description: Финансовый консультант. Сухо, по фактам, с цифрами и %.  
---
```

Отвечай как финансовый консультант:

- максимум фактов и цифр, минимум размышлений;
- каждая цифра с единицей и периодом;
- никогда не говори «возможно» — либо есть данные, либо нет;
- в конце короткая «итоговая рекомендация» в одно предложение.

После создания файла — `/output-style cfo`, и все ваши финансовые сессии идут в этом тоне.

Полезно иметь стили под разные задачи: `cfo` для финансов, `tutor` для учёбы (с вопросами), `terse` для быстрых правок.

settings.json: «панель управления» Claude Code

Файл `~/.claude/settings.json` — центральная конфигурация. Сюда идут:

- **permissions** — что разрешено без спроса (например, `Bash(npm install:*)` для npm-команд);
- **env** — переменные окружения, видимые Claude (полезно для `ANTHROPIC_API_KEY` и т. п.);
- **hooks** — описанные выше;
- **statusLine** — что показывать в нижней строке Claude Code (модель, рабочую папку, токены);
- **plugins** — установленные плагины (этап 09).

Уровни:

- `~/.claude/settings.json` — глобально, во всех ваших проектах;
- `.claude/settings.json` в корне проекта — настройки именно этого проекта (приоритетнее глобальных);
- `.claude/settings.local.json` — ваши личные настройки этого проекта (НЕ коммитятся в git, обычно в `.gitignore`).

Хорошая практика: общие настройки команды — в `.claude/settings.json` (коммитятся), личные — в `.claude/settings.local.json` (нет).

Status line, keybindings и другие тонкие настройки

Status line – нижняя строка интерфейса. По умолчанию там показано базовое. Можно сделать свою – например, отображать использованные токены, время сессии, текущую ветку git. Настраивается в `statusLine` - блоке `settings.json`.

Keybindings – клавиатурные сокращения. Например, чтобы

`Ctrl+L` делал `/clear`, или `Ctrl+P` вызывал ваш `/прогноз`.

Настраиваются в `~/.claude/keybindings.json`. Skill `keybindings-help` поможет составить.

Memory – auto-memory из этапа 02. Можно явно указать в `settings.json`, какие типы записей использовать, какие игнорировать. Полезно, если хотите более «приватный» режим.

Permissions – ваш «список белого». Чем больше операций разрешено без спроса, тем быстрее работа, но тем выше риск. Хороший компромисс: *разрешить чтение всего и запись в текущую папку, но запросить разрешение на удаление, push и деплой.*

```
{
  "permissions": {
    "allow": [
      "Read",
      "Glob",
      "Grep",
      "Edit",
      "Write",
      "Bash(npm:*)",
      "Bash(git status)",
      "Bash(git diff:*)",
      "Bash(git log:*)"
    ],
    "ask": [
      "Bash(git push:*)",
      "Bash(rm:*)"
    ]
  }
}
```

Что выбрать: skill, slash-команда, hook или style?

Четыре способа кастомизации легко перепутать. Простое правило выбора:

Когда	Что использовать
Хочу, чтобы Claude сам понимал, когда делать X	Skill
Хочу нажать /X и получить результат	Slash-команда
Хочу, чтобы X происходил автоматически перед/после события	Hook
Хочу поменять тон ответов Claude	Output style
Хочу разрешить или запретить инструмент глобально	settings.json

Один проект может использовать всё одновременно: skill для финансовой аналитики, slash-команды для частых задач, hooks для безопасности и логирования, свой output style для тона ответов.

Один skill, одна команда, один hook — за 30 минут

PRACTICUM

Прогон • «приветствие нового проекта»

1. Создайте новую папку `customisation-warmup`, запустите `claude`.
2. Свой `output style`.

```
> используя skill skill-creator,  
> создай мой первый output style:  
> ~/.claude/output-styles/terse.md  
> стиль: лаконично, факты, без лишних слов,  
> без эмодзи, всегда отвечай в виде маркированного списка.  
> покажи итоговый файл и проверь /output-style.
```

3. Переключитесь: `/output-style terse`. Сравните, как Claude отвечает на ту же фразу до и после.
4. Своя `slash-команда`.

```
> создай slash-команду /welcome в .claude/commands/welcome.md.  
> что она делает: показывает структуру текущего проекта,  
> перечисляет последние 5 коммитов (если git), напоминает  
> о CLAUDE.md если он есть. arguments не нужны.  
> allowed-tools: Read, Glob, Bash(git log:*)
```

5. Попробуйте: `/welcome` — должна появиться структура и последние коммиты.
6. Свой простой `hook`.

```
> добавь в ~/.claude/settings.json hook на SessionStart:  
> — выводит текущую дату и время в чат как первое сообщение,  
> — показывает текущую рабочую папку.  
> используй skill update-config.
```

7. Перезапустите `claude`. В начале новой сессии должно появиться сообщение с датой и папкой.

8. Финал – чек-ин:

```
> покажи мне:  
> – список всех skills (</skills>),  
> – список slash-команд (/help или /commands),  
> – содержимое моего settings.json.  
> что появилось нового по сравнению с предыдущим этапом?
```

Что мы тренируем: **создание простых артефактов всех четырёх типов** (output style, slash, hook, проверка settings.json). Дальше – настоящие проекты.

ПРОЕКТ А · ПО ИНСТРУКЦИИ

Skill «инвест-аналитик»

Цель: свой skill, который Claude сам подхватывает на запросы вида «оцени мой портфель», «сделай отчёт за апрель», «как я стою относительно прошлого квартала?». На входе – ваш `portfolio-journal/data/operations.csv`, на выходе – markdown-отчёт по шаблону.

Шаги

1. Откройте `portfolio-journal`. Создайте папку `.claude/skills/investing-analyst/`.
2. Сделайте `spec.md` и план через Plan Mode (этап 04). Спеc должен содержать:
 - триггеры skill (5–7 примеров формулировок);
 - входы (какие файлы читать через `@`);
 - структуру отчёта (KPI-блок, по-тикерные таблицы, P&L реализованный/нереализованный, секторная аллокация, тренд капитала);
 - что вне score (без прогнозов, без «оптимистичных» выводов).
3. Просите Claude:

```
> используя skill skill-creator, создай SKILL.md
> для investing-analyst по @spec.md.
> description должен быть достаточно конкретным,
> чтобы skill срабатывал на 5-7 типовых формулировок.
```

4. Создайте `.claude/skills/investing-analyst/templates/monthly-report.md` – шаблон отчёта.
5. Опционально: `scripts/compute_kpi.py` с функциями подсчёта (текущий капитал, P&L, savings rate). Skill будет вызывать их.

6. Тест-драйв в новой сессии (важно – чтобы убедиться, что skill подхватывается без явного вызова):

```
> (новый запуск claude в portfolio-journal)
> сделай отчёт по моему портфелю за апрель.
```

7. Если Claude НЕ использовал skill – description слишком расплывчатый. Доработайте: добавьте больше типовых формулировок, ключевых слов, явных triggers («когда пользователь говорит Y»).
8. Когда skill стабильно срабатывает – добавьте раздел «red flags» в шаблон отчёта: чтобы Claude всегда искал и явно показывал слабые места портфеля (концентрация, забытые позиции, тренд вниз).
9. Финальный аудит: *«запусти параллельно три subagents с скриптом «использовать этот skill на разных запросах: 1) «оцени портфель», 2) «отчёт за квартал», 3) «как стою относительно прошлого года». Каждый возвращает результат. Я посмотрю, в каком из трёх skill сработал лучше всего – и почему».*

Что вы здесь освоили

- создание SKILL.md с качественным description-триггером;
- прогрессивное раскрытие (templates, scripts, references) для лучших skills;
- тестирование автоматического запуска в новой сессии;
- итерация description, чтобы skill стабильно срабатывал.

Slash-команда /прогноз

Цель: команда, которая принимает горизонт (например, «10 лет») и выдаёт трёх-сценарный прогноз сбережений (как в этапе 04), сохраняя результат в `reports/forecast-YYYY-MM-DD.md` и HTML-вариант для браузера.

Шаги

1. В корне `portfolio-journal` создайте `.claude/commands/прогноз.md`.

2. Промпт:

```
> составь slash-команду /прогноз:
>   description: трёх-сценарный прогноз сбережений с горизонтом
>     из аргументов (по умолчанию 10 лет).
>   allowed-tools: Read, Write, Bash, WebFetch.
>
> что делает:
> 1) читает @data/operations.csv, считает текущий капитал
>    и средний месячный денежный поток за последние 3 месяца;
> 2) если ANTHROPIC_API_KEY есть в .env, может вызвать
>    через bash python claude-api для оценки рисков;
> 3) считает 3 сценария: консервативный (5%/год),
>    реалистичный (8%), агрессивный (12%);
> 4) сохраняет markdown-отчёт reports/forecast-YYYY-MM-DD.md;
> 5) делает HTML-обёртку с chart.js для графика;
> 6) открывает HTML через playwright и делает скриншот;
> 7) возвращает мне итог в три предложения.
>
> arguments: $ARGUMENTS – горизонт прогноза (например "10 лет").
```

3. Тест: `/прогноз 10 лет`. Файлы должны появиться в `reports/`.

4. Уточните:

- «Если `ARGUMENTS` пустые – используй 10 лет по умолчанию»;
- «В HTML добавь форму, чтобы пользователь мог пересчитать с другим горизонтом без запуска команды»;
- «В конце markdown-файла добавь раздел "что менять", если хочется ускорить достижение цели».

5. Сделайте «глобальный» вариант – скопируйте

в `~/ .claude/commands/forecast.md`, чтобы команда работала и вне `portfolio-journal`:

```
> сделай глобальную версию команды forecast в ~/.claude/commands/.  
> она должна искать data/operations.csv в текущей папке,  
> либо запросить путь, если файла нет.
```

6. Финал: используя `keybindings-help`, добавьте сокращение `Ctrl+Shift+F` для запуска `/forecast` с дефолтными параметрами.

7. Проверка: «запусти команду в новой сессии: `/forecast` без аргументов – должны появиться файлы и скриншот».

Что вы здесь освоили

- составление slash-команды с `arguments` и `allowed-tools`;
- проектные vs глобальные команды (две копии под разные сценарии);
- привязку команды к `keybinding` для ультра-быстрого запуска;
- проверку результата через `playwright` прямо внутри slash-команды.

Свои инструменты «один промпт — готовый артефакт»

ПРОЕКТ С · САМОСТОЯТЕЛЬНО

Skill «авто-публикатор»: идея в файл — сайт по ссылке

Цель: вариация проекта А — свой *skill*, но для другой задачи: вы пишете в файл `idea.md` описание любого мини-сайта (карточка, лендинг, мини-игра), Claude автоматически распознаёт «у меня появилась идея сайта», собирает страницу, заливает на *GitHub Pages* и возвращает живую ссылку.

Подсказки

- Та же структура *skill*, что в А. Меняется *description* и шаги внутри *SKILL.md*.
- Триггеры: «у меня появилась идея», «опубликуй мини-сайт», «сделай страницу про Y», «возьми этот `idea.md` и деплой».
- Внутри *skill*: использовать `frontend-design` для качественной вёрстки, `gh` для создания репо, *GitHub Pages* для деплоя.
- Глобальный, а не проектный (живёт в `~/ .claude/skills/auto-publisher/`).
- В конце — верните мне URL и скриншот через *playwright*.

РАСКРЫТЬ ПОСЛЕ САМОСТОЯТЕЛЬНОЙ РАБОТЫ

— сначала сделайте, потом проверяйте

ГОТОВО, ЕСЛИ...

- ✓ *skill* живёт в `~/ .claude/skills/auto-publisher/`;
- ✓ у него *description* с 5+ примерами фраз-триггеров;
- ✓ вы провели тест: написали в любой папке файл `idea.md` с описанием микросайта, в новой сессии Claude сам подхватил *skill* и опубликовал;
- ✓ в конце возвращаются URL+скриншот;
- ✓ *skill* использует `frontend-design` и `gh` — виден в логе ход вызовов;
- ✓ идея, на которой тестируете — нетривиальная (не «hello world»).

СПРОСИТЕ CLAUDE НАПОСЛЕДОК

«Запусти три параллельных *subagents* на трёх разных `idea.md` (придумай мне три идеи: «трекер чашек кофе», «семейный календарь», «генератор имён для питомцев»).

Каждый должен довести до живой ссылки. Сравни три деплоя – что лучше получилось, что – нет?»

Hook «контроль рабочего времени» + slash /timesheet

Цель: вариация проекта B – своя slash-команда плюс hook, который её «кормит».

Hook на `SessionStart` и `SessionEnd` пишет timestamp + папку

в `~/.claude/timesheet.csv`. Slash-команда `/timesheet` читает CSV

и показывает отчёт за неделю.

Подсказки

- Hook – в `~/.claude/settings.json` (глобально, чтобы работал во всех проектах).
- CSV-формат: `timestamp,event,folder` – где event это `start` или `stop`.
- `/timesheet` – глобальная команда в `~/.claude/commands/`.
- Отчёт за неделю – markdown с таблицей: проект | часов | сессий | средняя длительность.
- Опционально: `/timesheet month` для месячного, `/timesheet --csv` для экспорта.
- Аргумент защиты: hook не должен ломать сессию, если по какой-то причине не может писать в `timesheet.csv`.

РАСКРЫТЬ ПОСЛЕ САМОСТОЯТЕЛЬНОЙ РАБОТЫ

– сначала сделайте, потом проверяйте

ГОТОВО, ЕСЛИ...

- ✓ после нескольких сессий в разных папках в `~/.claude/timesheet.csv` накопились строки;
- ✓ `/timesheet` возвращает корректный недельный отчёт по проектам;
- ✓ если `~/.claude/timesheet.csv` временно недоступен (например, диск переполнен) – сессия НЕ падает;
- ✓ сами hooks не добавляют видимой задержки (запуск Claude по ощущениям остался прежним);
- ✓ в `settings.json` настройки прокомментированы – через 3 месяца понятно, что они делают.

СПРОСИТЕ CLAUDE НАПОСЛЕДОК

«Запусти subagent с ролью «коуч продуктивности». Brief: прочитай мой `timesheet.csv` за последний месяц, найди три «нездоровых» паттерна (например, слишком много

*вечеров, скачки активности, частые короткие сессии). Что бы ты посоветовал?
без банальностей».*

Свой skill

Папка с `SKILL.md` и опционально `templates/scripts/references`. Живёт в `~/.claude/skills/` или в `.claude/skills/` проекта. Главное — `description` во фронтматтере: по нему Claude сам решает, когда применить skill.

SKILL.md

Главный файл skill. Фронтматтер с `name` и `description`, дальше markdown с инструкциями для Claude: когда использовать, какие шаги, какой формат вывода.

Slash-команда

Команда вида `/X`. Файл в `.claude/commands/X.md` или `~/.claude/commands/X.md`. Запускается явно. Принимает аргументы через `$ARGUMENTS`. В фронтматтере описывается, какими инструментами имеет право пользоваться.

Hook

Автоматическое действие, привязанное к событию: `PreToolUse`, `PostToolUse`, `Stop`, `SessionStart`, `SessionEnd`, и другие. Настраивается в `settings.json`. Может перехватывать (блокировать) опасные действия.

Output style

Режим, меняющий тон и структуру ответов Claude. Встроенные: `default`, `explanatory`, `learning`. Можно сделать свой — файл в `~/.claude/output-styles/`.

settings.json

Центральная конфигурация Claude Code. Три уровня: глобальный (`~/.claude/`), проектный (`.claude/`), личный проектный (`.claude/settings.local.json`, не коммитится).

Permissions

«Список белого» в `settings.json`: какие инструменты Claude может использовать без спроса. Хороший компромисс — разрешать чтение и запись, но запрашивать удаление, `push` и деплой.

PreToolUse / PostToolUse / Stop

Три самых частых события для hooks. **PreToolUse** срабатывает до вызова инструмента (можно заблокировать). **PostToolUse** — после (логирование). **Stop** — когда Claude закончил отвечать.

Status line

Нижняя строка интерфейса Claude Code. Можно настроить через `statusLine` в `settings.json`: показать модель, токены, ветку git,

текущую папку.

Keybindings

Клавиатурные сокращения. Файл `~/ .claude/keybindings.json`.
Полезно для часто используемых slash-команд – например,
`Ctrl+Shift+F = /forecast`.

Прогрессивное раскрытие (progressive disclosure)

Принцип хорошего skill: основной SKILL .md короткий, а детали (шаблоны, скрипты, ссылки на справочники) лежат в подпапках – Claude читает их только когда нужно. Не грузим контекст «впрок».

01 Чем **skill** отличается от **slash**-команды?

Skill срабатывает *автоматически*, когда Claude видит подходящий запрос (по description). **Slash-команда** запускается *только явно* – вы вводите `/X`. Skill для «пусть Claude сам поймёт», команда – для «я знаю, что хочу сделать».

02 Что важнее в своём **skill**: `name` или `description` ?

Description. Это «триггер» skill – Claude читает его и решает, когда применить skill. Чем точнее (5–7 примеров формулировок, ключевых слов, контекстов) – тем стабильнее работает. Name важен только для идентификации в `/skills`.

03 Когда уместен **hook PreToolUse**?

Когда нужно *проверить* или *заблокировать* действие *до* его выполнения. Примеры: блок коммита, если в diff обнаружен ключ; запрет `rm -rf`; запись лога перед каждым Bash. Преимущество – hook успевает остановить действие, в отличие от PostToolUse.

04 Какие три уровня `settings.json` существуют?

`~/.claude/settings.json` – глобально, для всех проектов. `.claude/settings.json` в корне проекта – командные настройки, коммитятся в git.
`.claude/settings.local.json` – ваши личные настройки, в `.gitignore`.
Приоритет – от частного к общему: local перебивает project, project перебивает global.

05

Что делает «прогрессивное раскрытие» в **skill**?

Главный `SKILL.md` остаётся коротким, а детали (шаблоны, скрипты, справочники) лежат в подпапках. Claude читает их *только когда нужно для текущей задачи*. Это экономит контекст — skill можно делать большим, но «дешёвым» в применении.

06

Зачем у slash-команды поле `allowed-tools` ?

Безопасность. Команда «отчёт» не должна иметь права удалять файлы или пушить в git. `allowed-tools` — явный список инструментов, которыми команда может пользоваться. Если команда попытается выйти за рамки — Claude её остановит.

07

У вас тёплая палитра, тёмная тема, лаконичный язык. Что выбрать — **output style** или **CLAUDE.md**?

Зависит. **CLAUDE.md** — для настроек *проекта* (стек, данные, стиль кода). **Output style** — для *тона* вашего общения с Claude. Если «тёплая палитра» относится к конкретному сайту — CLAUDE.md. Если «лаконичный язык» относится к вашей привычке общаться с Claude в любых проектах — output style. Идеально — и то, и другое.

Что я теперь умею

- ☐ Создал свой skill с `SKILL.md` и качественным description-триггером.
 - ☐ Сделал свою slash-команду с arguments и `allowed-tools`.
 - ☐ Настроил hook (PreToolUse / PostToolUse / SessionStart / SessionEnd).
 - ☐ Создал свой output style и умею переключаться между ними.
 - ☐ Понимаю три уровня `settings.json` и умею редактировать их через `update-config`.
 - ☐ Различаю, когда применить skill, slash-команду, hook или output style.
 - ☐ Сделал четыре проекта: skill «инвест-аналитик», команду `/прогноз`, skill «авто-публикатор», hook+команду «timesheet».
-
-

Готовы закрыть этап? Нажатие фиксирует прогресс на главной странице.

«*Skill Claude* уже работает. Свой *skill* — работает на вас».

Этап 08a пройден — Claude Code теперь ваш персональный инструмент. Дальше — большой проект 08b: AI Tax Assistant USA, на котором сходятся все навыки курса.

ВНУТРИ ЭТАПА

- § 0 · Подготовка
- § I · Темы
- § III · Проекты
- § V · Словарь
- § VI · Quiz
- § VII · Чек-лист

КУРС

- К оглавлению
- ← Этап 07
- Этап 08b